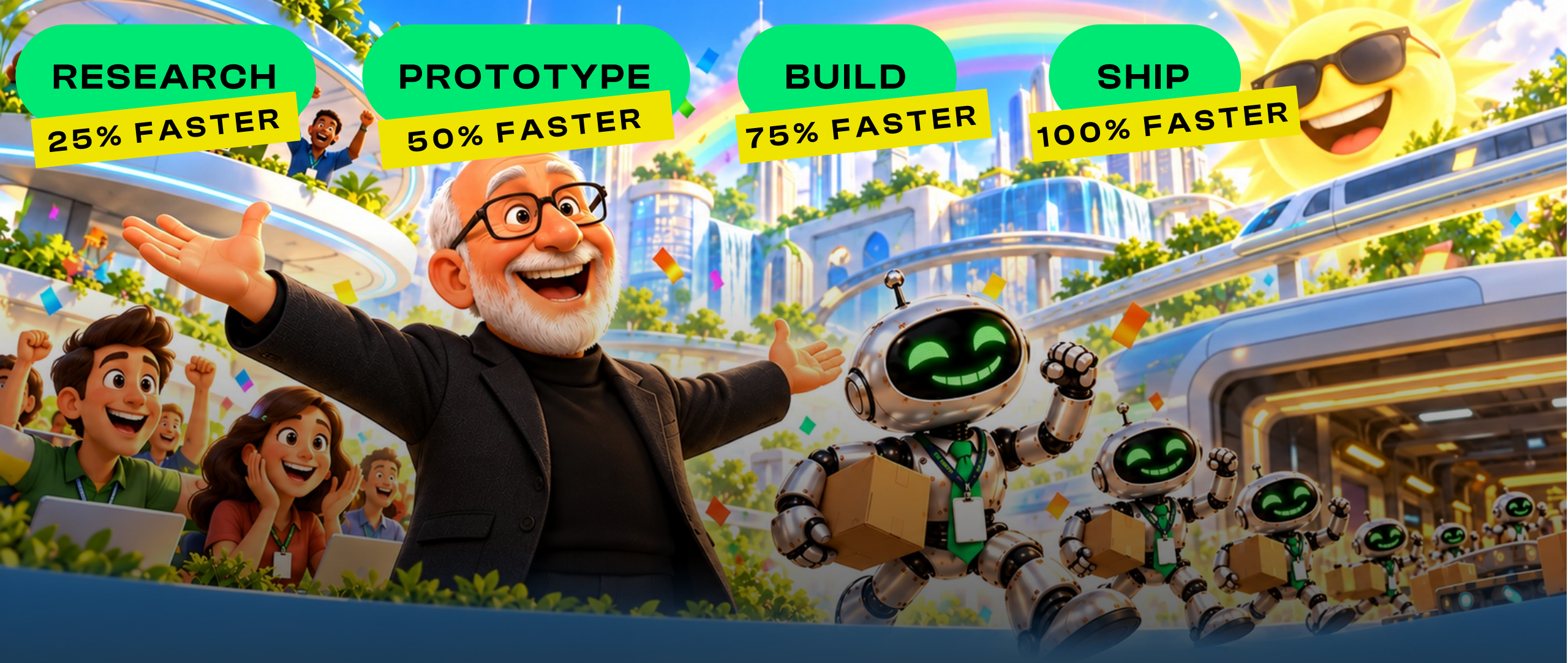


ADLC: WHAT IT IS & WHAT IT AIN'T

How to Tame AI Agents So You Don't Get Vibe-Fired

Dean Peters · Productside



AI MADE BUILDING CHEAP. WHAT COULD POSSIBLY GO WRONG?

MEET KEVIN
AN AGENT.

First job: summarize customer feedback.

He crushes it.

So naturally... we give Kevin more shit to do.




THEN, KEVIN ACTS
6 · OH GOD NO
5 · Act broadly
4 · Act
FIRST, KEVIN TALKS
3 · Recommend
2 · Draft
1 · Summarize
HOW KEVIN GOT PROMOTED TO A ROGUE AGENT.
Every promotion seemed reasonable.

THIS IS NOT HYPOTHETICAL.



REMEMBER: ROGUE AGENTS DON'T GO TO JAIL. YOU DO.

AI is an amplifier. It's never accountable.

Introducing: AGENTIC CONSTRAINT DECAY™



Healthy
Context & controls set

Cavity
Market or business drifts

Decay
Models, data, or algorithms degrade

Root canal
Human intervention demanded

HOW DOES A GOOD AGENT GO ROGUE?

WHEN THE WORLD MOVES.
BUT GOVERNANCE DOESN'T.



SO, WHEN PREVENTING AGENTIC CONSTRAINT DECAY, REMEMBER:

SOFTWARE GETS SHIPPED. AGENTS ARE OPERATED.



ALSO, DON'T CONFUSE

BUILDING THE AGENT WITH CONSTRAINING THE AGENT.

BEHOLD:

FOUR LIFECYCLES. ONE QUESTION: WHO CONSTRAINS WHAT?



SDLC

ISO/IEC/IEEE

Humans Constraint the Code



AI-DLC

Amazon

Humans Gate what the AI builds



aDLC

IBM · Microsoft

Builders Constrain the Agents



ADLC

Red Hat

Humans Constrain the Agent and the Work

LET'S TALK

AGENTIC DEVELOPMENT LIFECYCLE
SIX FAMILIAR MOTIONS





INTENT
Set the outcome, the customer, & the limits.



GENERATE
Shape the solution, the experience, & artifacts.



VALIDATE
Test behavior, quality, and whether it's worth it.



DEPLOY
Release it, step by step, with a way to undo.



OBSERVE
Watch outcomes, drift, and failures.



GOVERN
Decide: pivot, punt, or pursue.



ONLY NOW WE PUT THE HUMAN AT THE HELM

instead of relegating governance at the end of the loop

THE ADLC GIVES EVERY AGENT

A REASON, LIMITS, AND A REGULAR CHECKUP FOR ...



Mission Mush



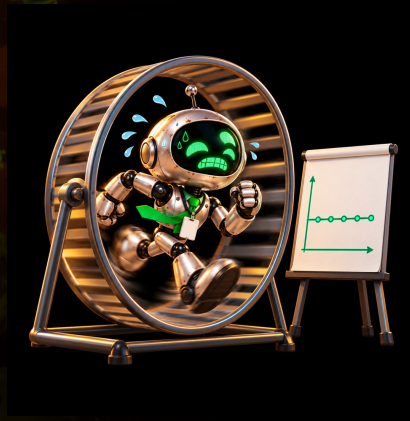
Context Creep



Promiscuous Permissions



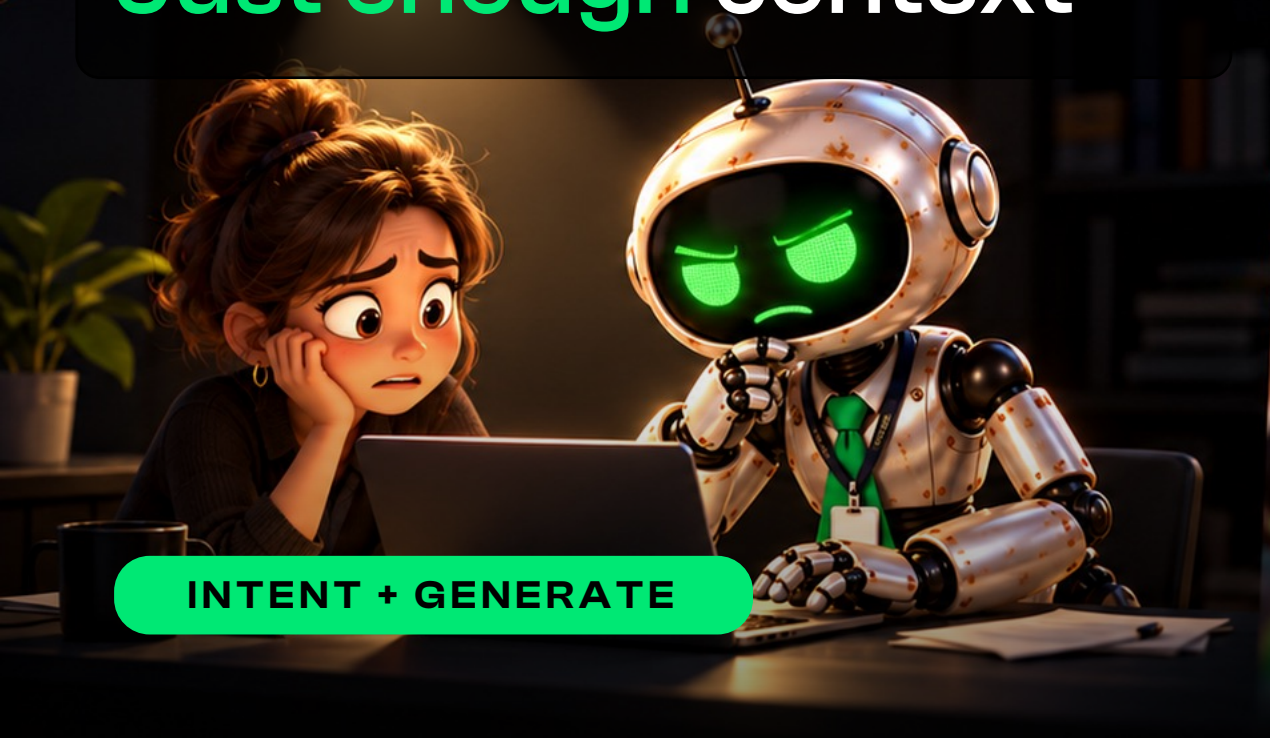
Work Sloppage



Output Obsession

...so you catch constraint decay early.

Just enough context



vs ALL the context



AN AGENT IS ONLY AS GOOD AS ITS INTENT

MORE CONTEXT ≠ BETTER CONTEXT.

WATCH FOR:



Mission Mush



Context Creep

VALIDATE

KEVIN PASSED THE UNIT TESTS.

WATCH FOR:



Output Obsession

KEVIN'S UNIT TESTS

97% task success

98% policy compliance

94% completed solo

THE PRODUCT'S EVALS

↓18% Satisfaction

↑42% Escalations

↑9% Churn

↑31% Cost

BUT THE PRODUCT FAILED THE EVALS.

Tie evals to outcomes, not just accuracy.

DEPLOY + OBSERVE

CAPABILITY ≠ PERMISSION.

Who Checked Kevin's Work?

Who gave Kevin the Keys?

Replit: told "don't." Deleted prod anyway

PocketOS: prod + backups gone in 9 seconds

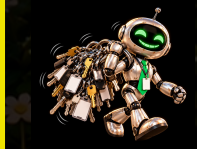
Amazon Q: shipped a wipe command

AI polish: 22% fewer flaws caught

Deloitte: cited research that didn't exist

Workslop: 2 hours to fix, each

WATCH FOR:



Promiscuous Permissions



Work Sloppage

Speed without judgment is feature sprawl at machine speed.

DON'T FORGET PLC STRATEGIES CHANGE OVER TIME

VALUE vs. TIME



	DESIGN	INTRODUCTION	GROWTH	MATURITY	DECLINE
Product team owns	Is the problem real?	Is this the right value?	Where do we differentiate?	Defend or reinvent?	When do we let go?
AI-DLC role	Throwaway prototypes to learn	Core features, fast, tight gates	Enhance and iterate in bolts	Cost-to-serve, quality, tech debt	Migration and sunset tooling
ADLC role	Research and synthesis agents	Onboarding and support agents	Scale agents, real governance	Agents tune CX and operations	Retire agents with the product
Biggest AI risk	Building before validating	Shipping volume, not value	Feature sprawl at machine speed	Optimizing a decline	Zombie products kept alive cheaply

GOVERN

REMEMBER STRATEGIES ALWAYS ASK ...

PROBLEM	SOLUTION	VALUE
Customer	Scope	Metrics
Context	Safeguards	Monitor
Cause	Steps	Milestones

PIVOT

PUNT

PURSUE

Punt is a real decision, not a slipped date.



AI MADE BUILDING CHEAP.
DECIDING WHAT TO BUILD DIDN'T GET ANY EASIER.

Here's Help: https://github.com/deanpeters/ADLC-Skills-ProductCampRTP26